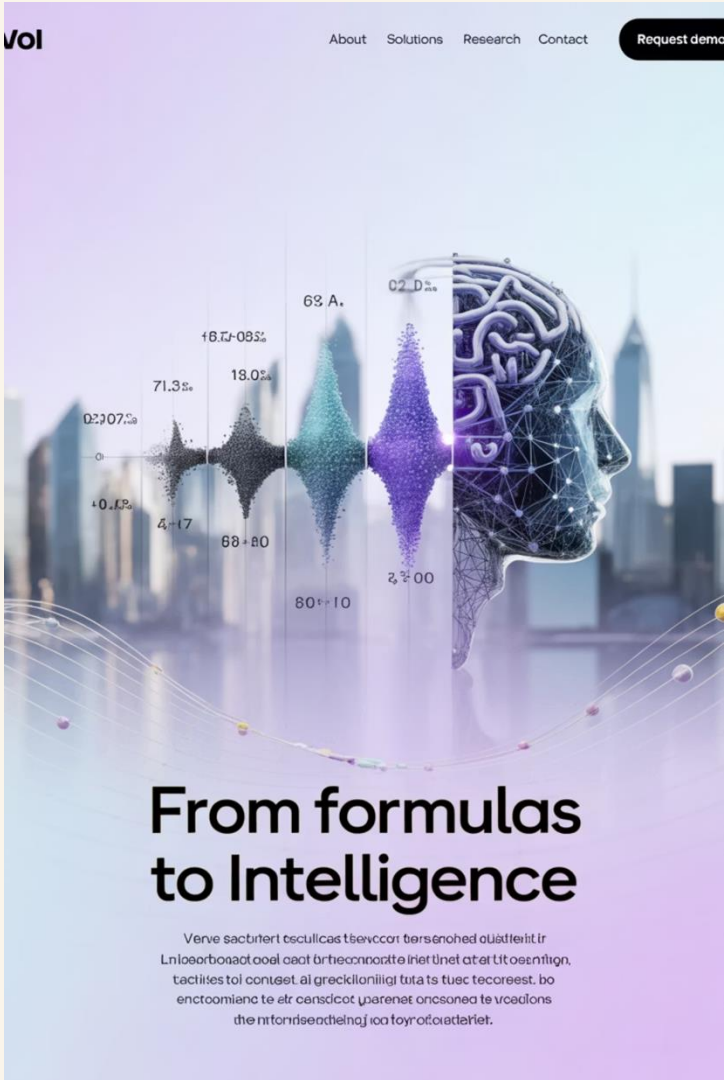
Fine-Tune LLMs

From mathematical models to generative AI

Andy Li, PhD, CFA, FRM



Guest Speaker’s bio

Alex Zhang has over 15 years of experience in structured credit and quantitative investing. Most recently, he served as Portfolio Manager and Head of Structured Credit at Squarepoint Capital in New York. Since joining the firm in 2020, Alex built and led the structured credit platform from inception, overseeing investments across CLOs, non-agency RMBS, CMBS, ABS, and SRT strategies. He was responsible for portfolio management and trading, while also leading the development of quantitative research, pricing models, and the analytical infrastructure supporting the platform.

Prior to Squarepoint, Alex was a Structured Credit Trader at Guggenheim Partners. Earlier in his career, he held portfolio management and trading roles at several hedge funds, focusing on structured products and quantitative investment strategies.

Alex holds a Master of Financial Engineering from the University of California, Berkeley (Haas School of Business), a Master of Science in Mathematics from Northwestern University, and a Bachelor of Science in Mathematics from the University of Science and Technology of China.

Outline

- 🤔 Why Fine-tune LLMs?
- 🧠 What is Fine-tuning?
- 🔧 How to Fine-tune LLMs

Why Fine-tuning LLMs?


Input /
Prompt
LLM
Output /
Response

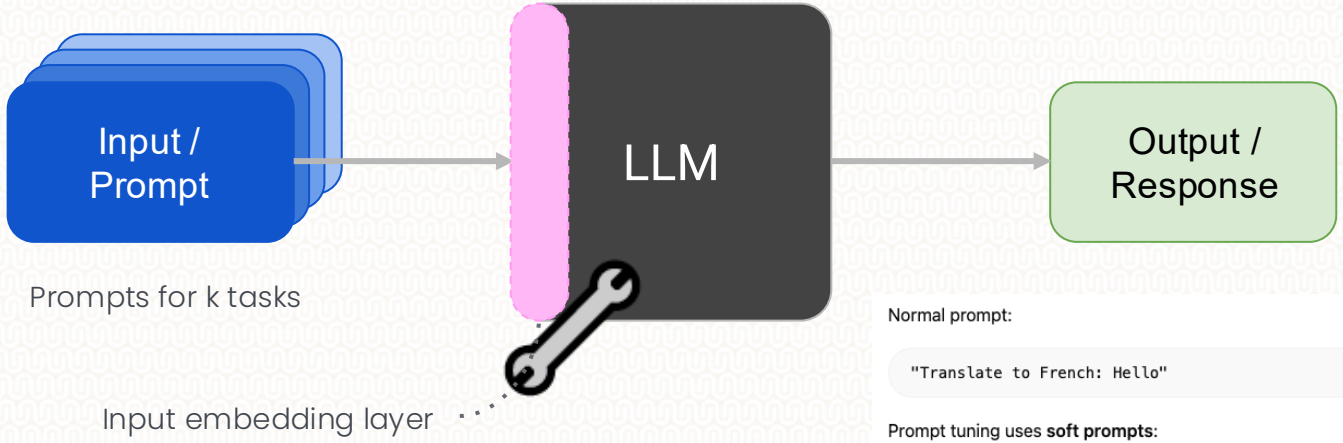
Prompt Engineering

Holding the model weights fixed and updating the input prompt to obtain the desired output.



Prompt Tuning

Holding a set of prompts for *k* tasks fixed while updating the model's input embedding layer.



Normal prompt:

```
"Translate to French: Hello"
```

Prompt tuning uses **soft prompts**:

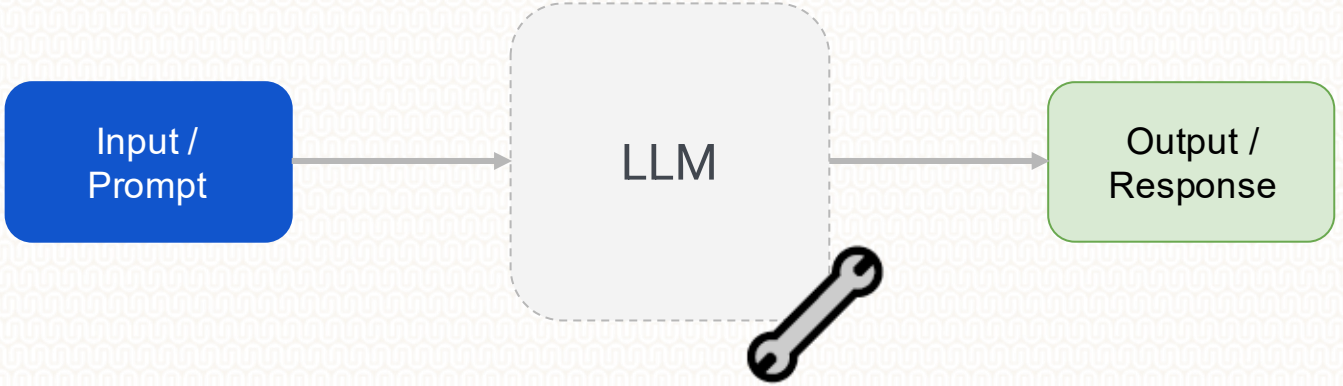
```
[p1 p2 p3 p4] + "Hello"
```

Where `p1 p2 p3 p4` are **learnable embedding vectors**, not words.

They exist **only inside the embedding layer**.

Fine-tuning

Updating model weights using a specific data distribution to obtain the desired behavior from the model.



Pros and Cons

Prompt Engineering

✅ Access to powerful proprietary models

✅ Adaptable to many tasks

✅ No ML/Data/Eng expertise needed

✅ Costs easily controlled

❌ Weights not accessible

❌ Limited fine-tuning capabilities

❌ Yields data to walled garden

❌ Subject to knowledge cut-offs

Fine-tuning

✅ Full access to model weights

✅ Can be specialized to specific tasks

✅ Control over training and inference data

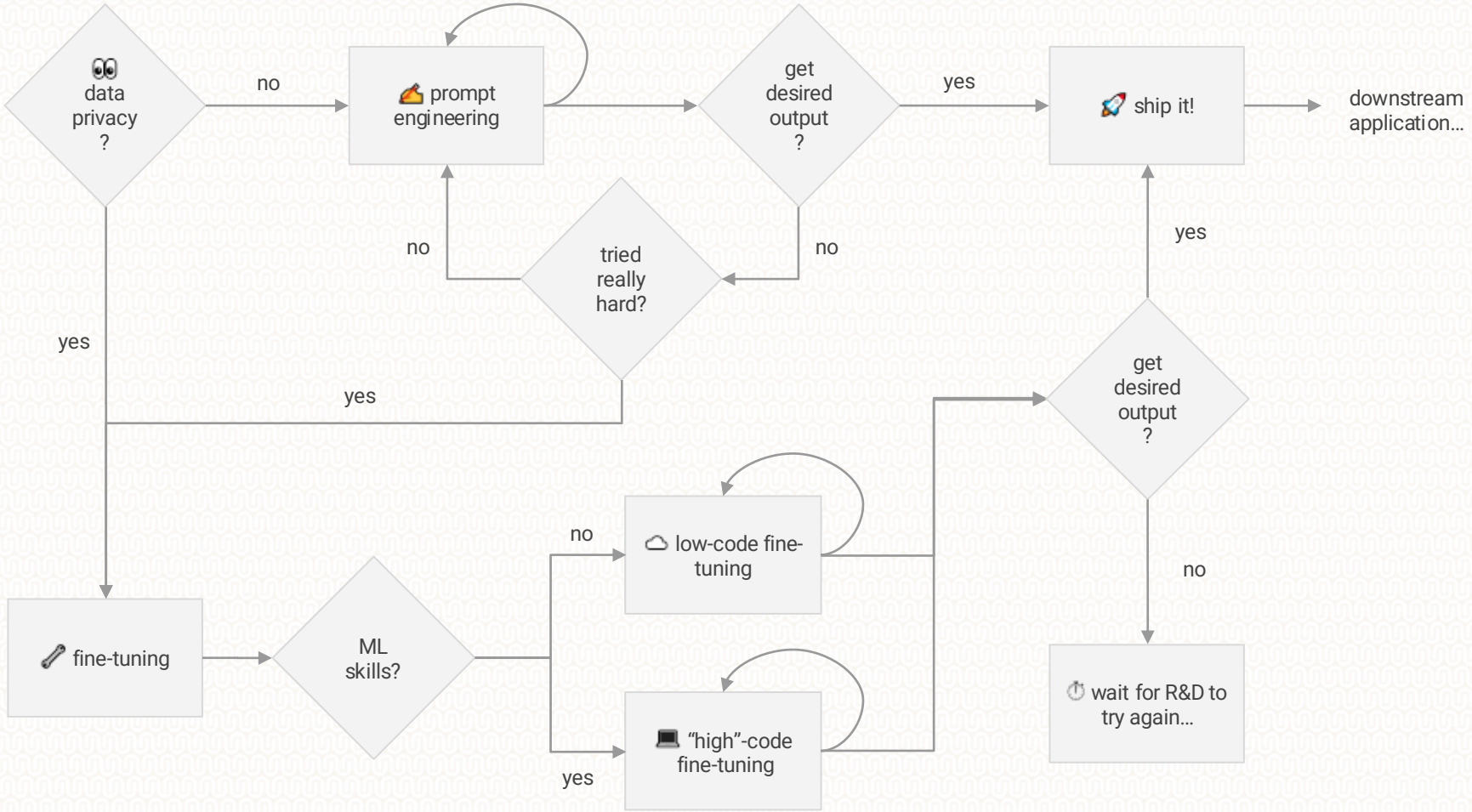
✅ Better than pre-training from scratch

❌ Requires ML/Data/Eng expertise

❌ GPUs are costly!

❌ Not as performant as proprietary models

❌ Potentially for no or little ROI


👀 data privacy ?
no
✍️ prompt engineering
get desired output ?
yes
🚀 ship it!
downstream application...
no
tried really hard?
no
yes
yes
yes
get desired output ?
yes
🩹 fine-tuning
ML skills?
no
low-code fine-tuning
no
💻 “high”-code fine-tuning
yes
⏱ wait for R&D to try again...

Prompt Engineering Resources

OpenAI:
https://platform.openai.com/docs/guides/gpt-best-practices/six-strategies-for-getting-better-results

Microsoft: https://learn.microsoft.com/en-us/azure/cognitive-services/openai/concepts/advanced-prompt-engineering?pivots=programming-language-chat-completions

Cohere:
https://docs.cohere.com/docs/prompt-engineering

Udemy:
https://www.udemy.com/course/prompt-engineering-for-ai

Coursera:
https://www.coursera.org/learn/prompt-engineering

👊 “Just” write instructions clearly! ✍️

RAG vs Fine-tuning

Feature Comparison	RAG	Fine-Tuning
Knowledge Updates	Directly updating the retrieval knowledge base ensures that the information remains current without the need for frequent retraining, making it well-suited for dynamic data environments.	Stores static data, requiring retraining for knowledge and data updates.
External Knowledge	Proficient in leveraging external resources, particularly suitable for accessing documents or other structured/unstructured databases.	Can be utilized to align the externally acquired knowledge from pretraining with large language models, but may be less practical for frequently changing data sources.
Data Processing	Involves minimal data processing and handling.	Depends on the creation of high-quality datasets, and limited datasets may not result in significant performance improvements.
Model Customization	Focuses on information retrieval and integrating external knowledge but may not fully customize model behavior or writing style.	Allows adjustments of LLM behavior, writing style, or specific domain knowledge based on specific tones or terms.
Interpretability	Responses can be traced back to specific data sources, providing higher interpretability and traceability.	Similar to a black box, it is not always clear why the model reacts a certain way, resulting in relatively lower interpretability.
Computational Resources	Depends on computational resources to support retrieval strategies and technologies related to databases. Additionally, it requires the maintenance of external data source integration and updates.	The preparation and curation of high-quality training datasets, defining fine-tuning objectives, and providing corresponding computational resources are necessary.
Latency Requirements	Involves data retrieval, which may lead to higher latency.	LLM after fine-tuning can respond without retrieval, resulting in lower latency.
Reducing Hallucinations	Inherently less prone to hallucinations as each answer is grounded in retrieved evidence.	Can help reduce hallucinations by training the model based on specific domain data but may still exhibit hallucinations when faced with unfamiliar input.
Ethical and Privacy Issues	Ethical and privacy concerns arise from the storage and retrieval of text from external databases.	Ethical and privacy concerns may arise due to sensitive content in the training data.

What is Fine-Tuning?

Fine-Tuning 101: Theoretical Foundations

What is Fine-Tuning?

Fine-tuning is the process of continuing training on a **pre-trained model** using a **small-scale specific dataset** to adapt the model to new tasks, domains, or styles.

Application Scenarios

1

Instruction Following

Models like Alpaca and ChatML adaptations

2

Classification

Sentiment analysis for movie/product reviews

3

Domain-Specific QA

Legal documents or financial filings analysis

4

Dialogue Optimization

Dedicated customer service or role-playing chatbots



How LLMs Learn

•**Pre-training:** The model is trained with self-supervised learning on large-scale, diverse text corpora to learn general linguistic representations, syntax, semantics, and world knowledge.

•**Supervised Fine-Tuning (SFT):** The pretrained model is fine-tuned on curated, task-specific datasets using supervised learning to improve performance on downstream tasks and instruction-following behavior.

•**Reinforcement Learning from Human Feedback (RLHF) / Preference Learning:** A reward model is trained from human preference data, and the language model is optimized using reinforcement learning (e.g., PPO) to align outputs with human judgments of quality, safety, and usefulness.

Pre-training

- Large language models are typically pre-trained on massive public datasets such as Common Crawl, which aggregates web content collected over many years.

- This large-scale exposure allows models to develop general language understanding, factual knowledge, and early reasoning abilities prior to task-specific refinement.

Supervised Fine-Tuning (SFT)

- Supervised Fine-Tuning (SFT) is a critical step in aligning a pretrained language model to real-world tasks and business needs.
- By training on carefully labeled examples (Input-Output pairs), the model is refined to produce more accurate, consistent, and task-specific outputs.
- Most fine-tuning efforts rely on labeled data, while unlabeled domain adaptation is used selectively in specialized scenarios.

Alignment Phase

- The alignment phase focuses on modifying the behavioral characteristics of a pretrained language model to better reflect human preferences, normative constraints, and safety objectives.

- In contrast to pre-training, which learns general linguistic representations, and model scaling, which improves capacity and generalization, alignment techniques explicitly shape model outputs to ensure consistent instruction-following, helpfulness, and adherence to ethical and policy constraints.

- This phase typically aims to optimize the model such that it produces responses that are not only contextually appropriate, but also aligned with human expectations, task-specific objectives, and established safety guidelines.

Parameter-Efficient Fine-Tuning (PEFT)

Parameter-Efficient Fine-Tuning (PEFT) methods adapt large language models to downstream tasks by training a small subset of additional or auxiliary parameters, while keeping the majority of the base model weights fixed.
By limiting the number of trainable parameters, PEFT significantly reduces computational and memory requirements and mitigates catastrophic forgetting associated with full-model fine-tuning.

Common PEFT approaches include:
• **LoRA (Low-Rank Adaptation):** Injects trainable low-rank updates into existing weight matrices, enabling task adaptation while freezing the original parameters.

• **Adapters:** Inserts lightweight, task-specific neural modules within transformer layers, preserving the pretrained backbone.

• **Prefix Tuning:** Optimizes continuous task-specific prefix vectors that are prepended to the model's internal representations.

• **Prompt Tuning:** Learns soft prompt embeddings applied at the input level, without modifying the model's internal parameters.

Fine-Tuning Methods Comparison

Approaches to Parameter-Efficient Fine-Tuning

1

Method	Parameter Count	Flexibility	Training Cost
Full Parameter Fine-Tuning	High	High	High (VRAM intensive)
Prompt Tuning	Low	Medium	Low
LoRA / PEFT	Extremely Low (few million)	High	Low (few hours)

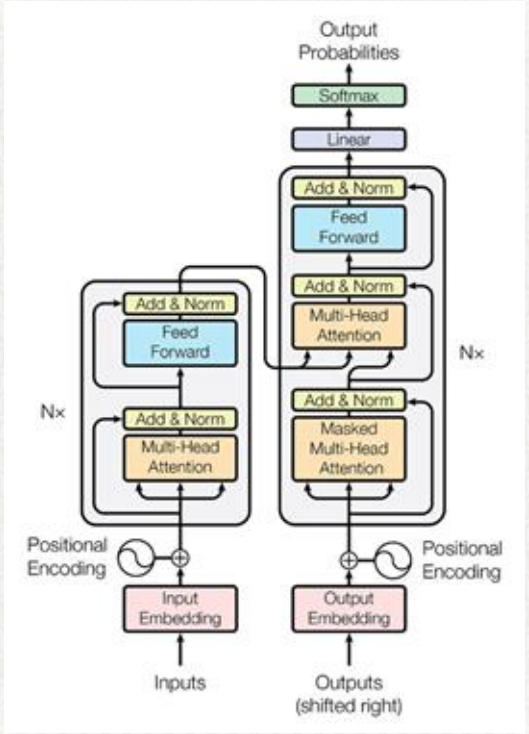
Parameter-efficient fine-tuning methods like LoRA provide comparable performance to full fine-tuning while requiring only a fraction of the computational resources.

How to Fine-tune LLMs

Transformers Recap

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

The attention mechanism represents a *collection of tokens* as **Queries**, **Keys**, and **Values**, allowing **information** to be passed between the tokens in successive layers.



Source: https://arxiv.org/abs/1706.03762

Transformers Recap

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

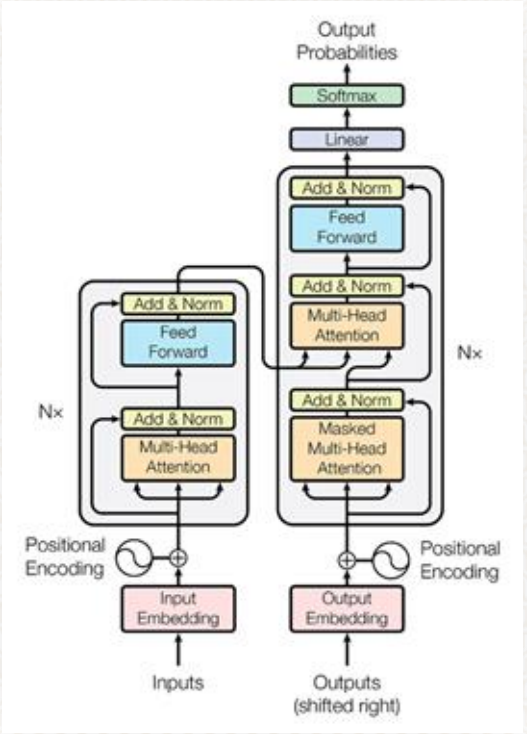
The result is that each *token's* value is **represented** and **re-represented** as a function of its **context** as more and more attention layers are stacked together.

Source: https://ai.googleblog.com/2017/08/transformer-novel-neural-network.html

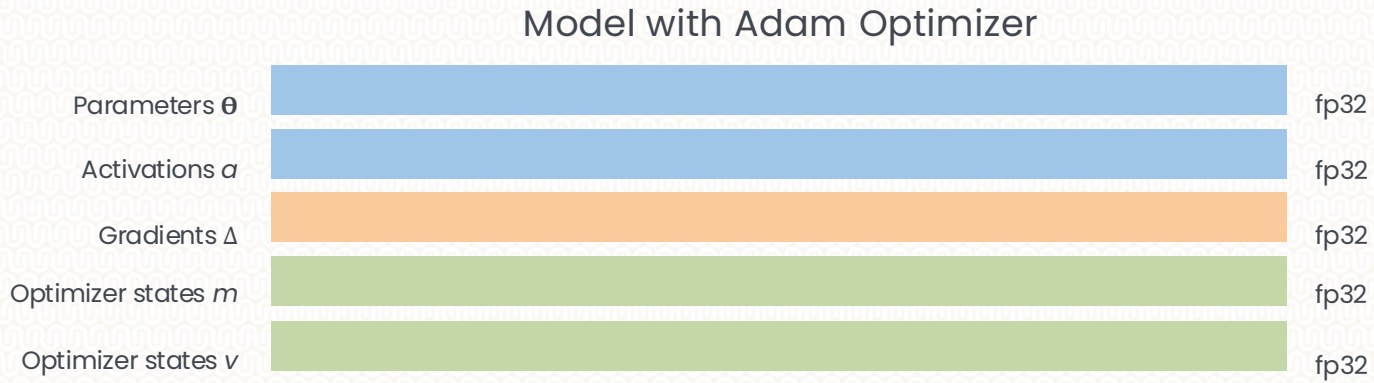
Transformers Recap

What's the problem with Attention?

- Require a lot of memory
- $O(n^2)$ complexity per attention layer
- Bound by fixed context length



How much memory do I need to train a model?



A model with 100 parameters needs 100 * 4 * 5 = 2,000 bytes of memory

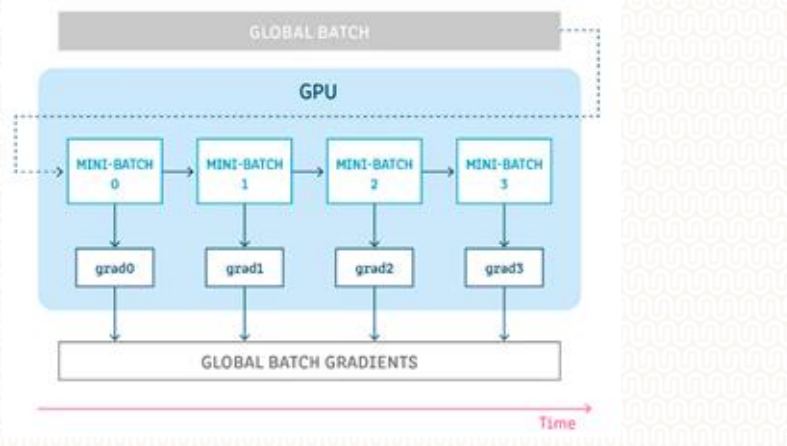
A model with 1B parameters needs 1e9 * 4 * 5 = 2e10 bytes (20 Gi) of memory

20x Multiplier

Source: https://www.microsoft.com/en-us/research/blog/zero-deepspeed-new-system-optimizations-enable-training-models-with-over-100-billion-parameters/

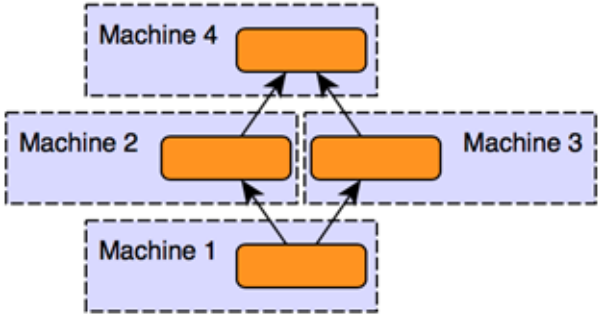
Methods to train large models

Gradient Accumulation



Source: https://aman.ai/primers/ai/grad-accum-checkpoint/

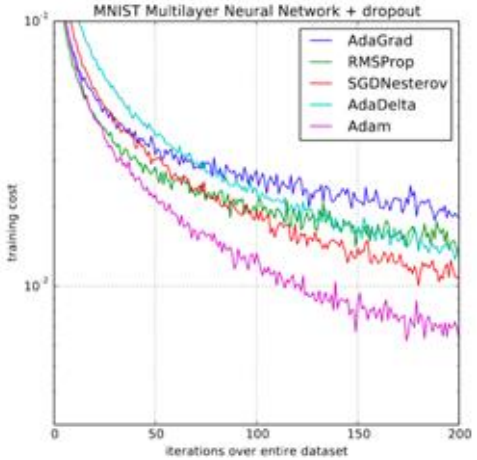
Model Parallelism



Source: https://xiandong79.github.io/Intro-Distributed-Deep-Learning

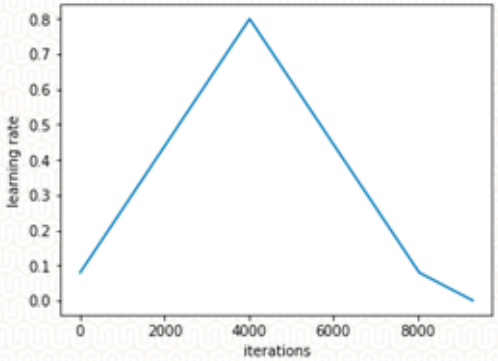
Methods to train models faster

Optimizers



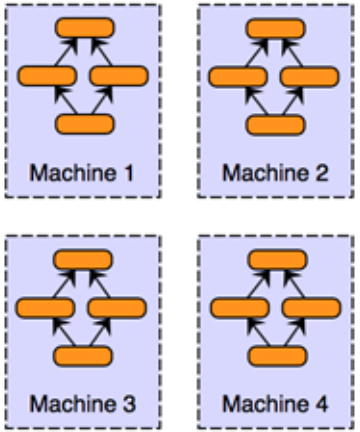
Source: https://www.fast.ai/posts/2018-07-02-adam-weight-decay.html

Schedulers



Source: https://github.com/sgugger/Deep-Learning/blob/master/Cyclical%20LR%20and%20momentums.ipynb

Data Parallelism

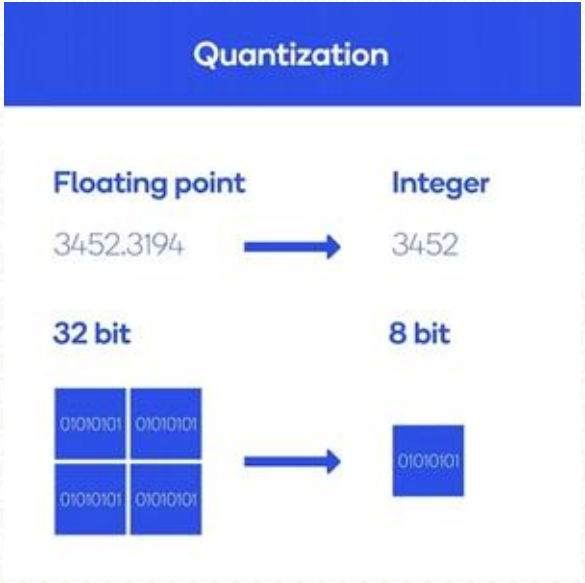


Source: https://xiandong79.github.io/Intro-Distributed-Deep-Learning

Intro to 8-bit Quantization

Quantization reduces memory requirements but trades off precision.

You can **fit larger models** into your GPUs, but it can lead to **training instability**.



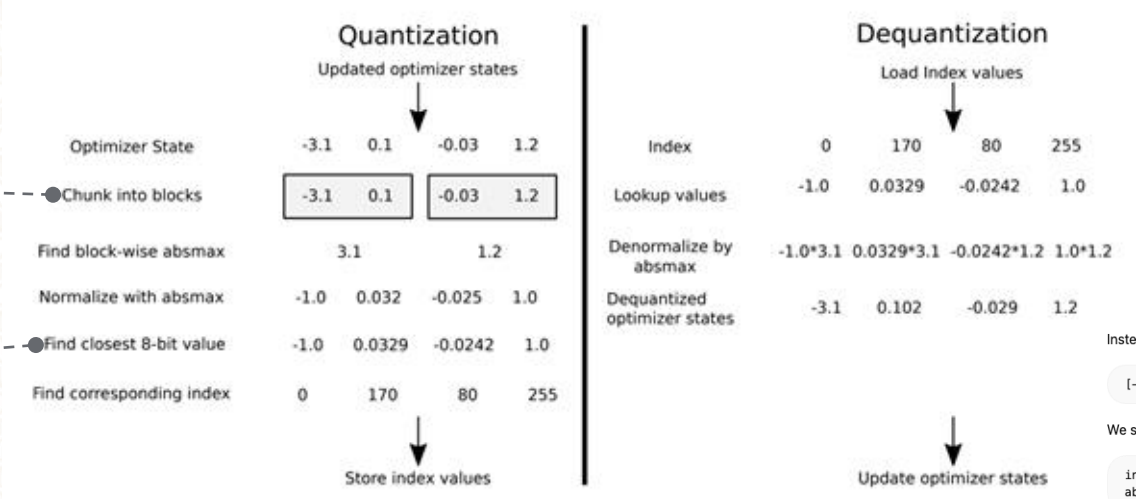
Problem: How to deal with outliers?

8-BIT OPTIMIZERS VIA BLOCK-WISE QUANTIZATION

Tim Dettmers[*‡] **Mike Lewis**[*] **Sam Shleifer**[*] **Luke Zettlemoyer**[*‡]
Facebook AI Research[*], {mikelewis, sshleifer}@fb.com
University of Washington[‡], {dettmers, lsz}@cs.washington.edu

Isolate outliers to particular blocks

Dynamic quantization preserves precision of small and large value



Instead of storing full precision numbers:

```
[-3.1, 0.1, -0.03, 1.2]
```

We store:

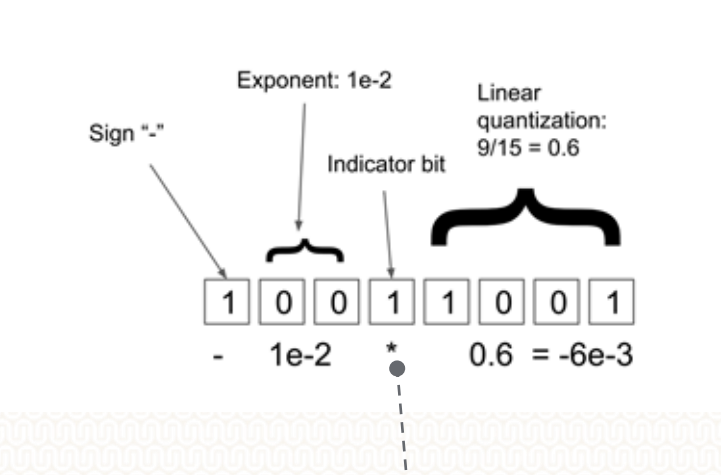
```
index values: [0, 170, 80, 255]
absmax values: [3.1, 1.2]
```

Very small memory footprint.

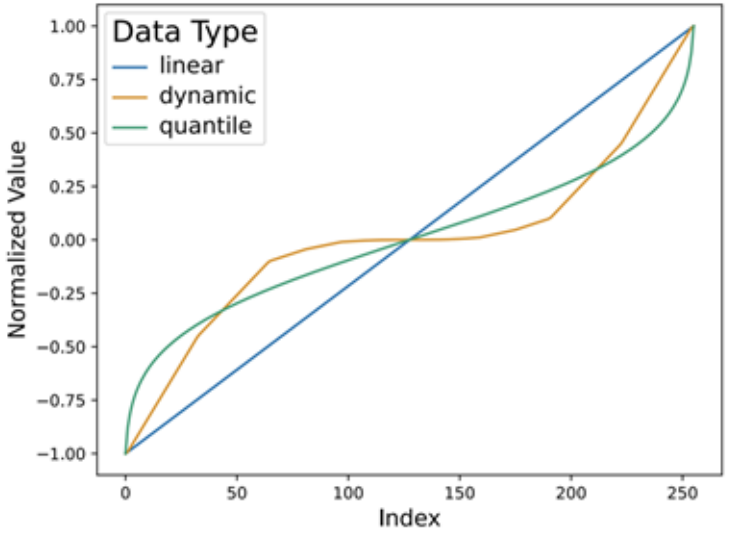
Source: https://arxiv.org/abs/2110.02861

Problem: How to deal with outliers?

Dynamic Tree Quantization



Make trade-off between high precision for large values or small values



Source: https://arxiv.org/abs/2110.02861

Why go through all this trouble?

From the paper:

"For **32-bit states**, Momentum and Adam consume **4 and 8 bytes** per parameter [respectively].

That is **4 GB** and **8 GB** for a 1B parameter model.

Our **8-bit non-linear quantization** reduces these costs to **1 GB** and **2 GB.**"

https://arxiv.org/abs/2110.02861

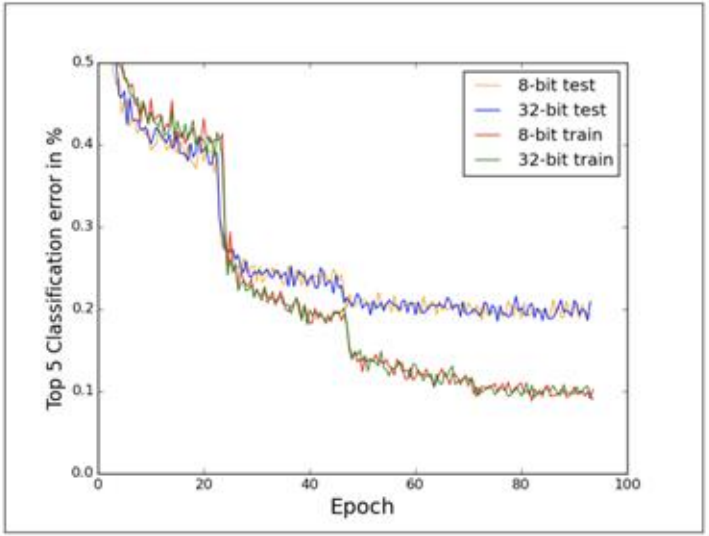


Figure 2: Classification train and test error for the 8-bit dynamic data type used in AlexNet on the ImageNet dataset.

Source: https://arxiv.org/abs/1511.04561

Low-Rank Adaptation (LoRA)

In regular fine-tuning, you have some weights W.

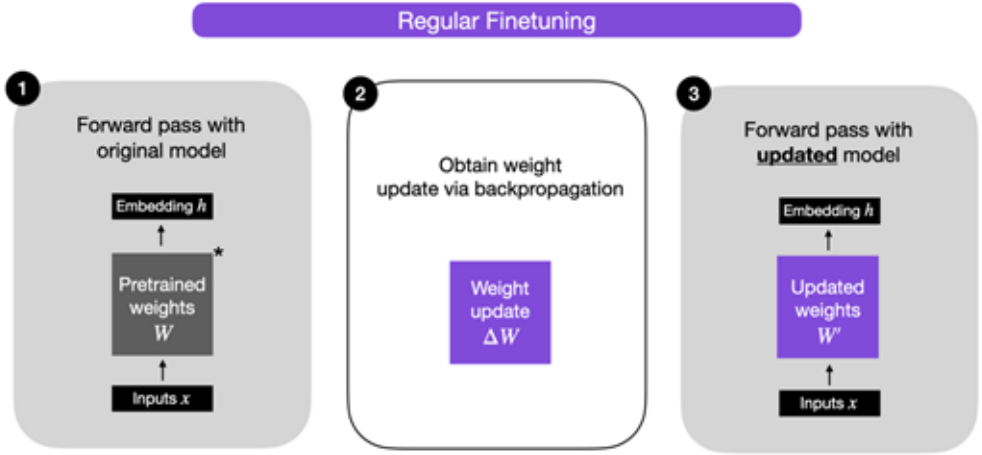
Backpropagation results in a weight update ΔW.

Updated weights a obtained by:

$W' = W + \Delta W$

Doing a forward pass with the new weights is given by:

$h = W'x$



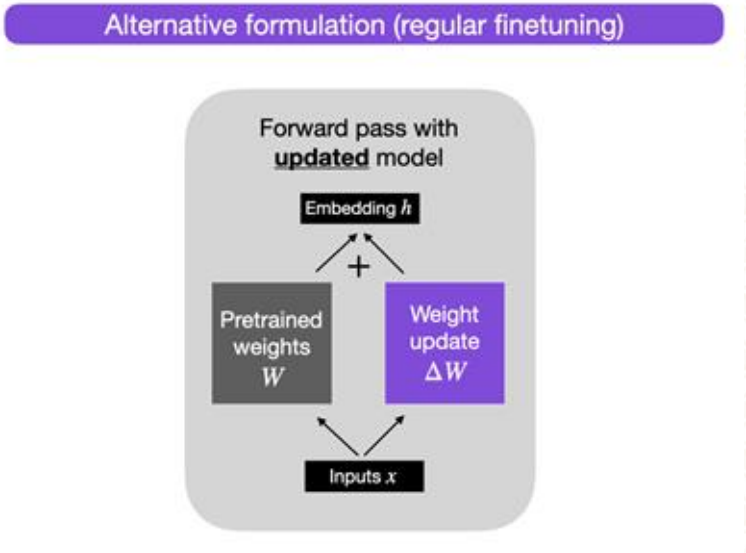
Source: https://sebastianraschka.com/blog/2023/llm-finetuning-lora.html

Low-Rank Adaptation (LoRA)

Alternatively, we can formulate the forward pass into a sum of the original weights * input and the updated weights * input:

$h = W'x + = Wx + \Delta Wx$

🤔But why would we do this?



Source: https://sebastianraschka.com/blog/2023/llm-finetuning-lora.html

Low-Rank Adaptation (LoRA)

Formulation

Decompose the weight updates for $W \in \mathbb{R}^{A \times B}$ into two smaller matrices:

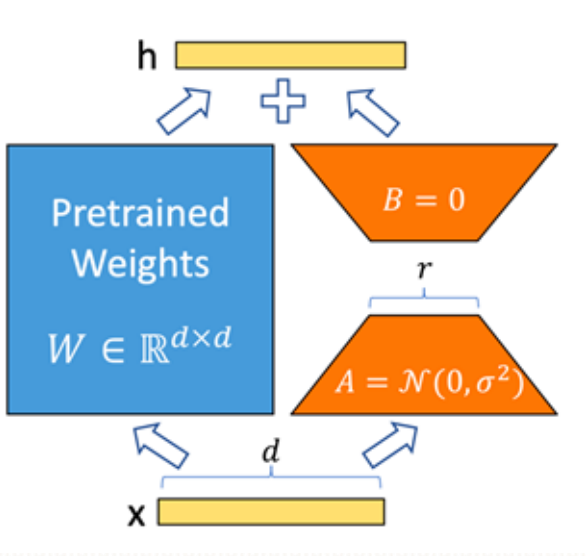
$\Delta W = W_A W_B$

Where: $W_A \in \mathbb{R}^{A \times r}$ and $W_B \in \mathbb{R}^{r \times B}$

This allows us to reformulate:

$h = W'x + = Wx + (W_A W_B)x$

r places an upper bound on the **rank** of the adaptation matrices, where the rank is the number of linearly independent rows/columns of a matrix.



Source: https://arxiv.org/abs/2106.09685

Low-Rank Adaptation (LoRA)

Hypothesis

During pre-training, $\Delta \mathbf{W}$ has high "intrinsic rank", i.e. most of the rows/columns of the matrix are linearly independent

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Extreme example: the identity matrix has full rank, i.e. all the rows and columns are linearly independent

During fine-tuning to a specific task, $\Delta \mathbf{W}$ has low "intrinsic rank", i.e. most of the rows/columns of the matrix are redundant

$$\begin{bmatrix} 1 & 0 & 1 \\ -2 & -3 & 1 \\ 3 & 3 & 0 \end{bmatrix}$$

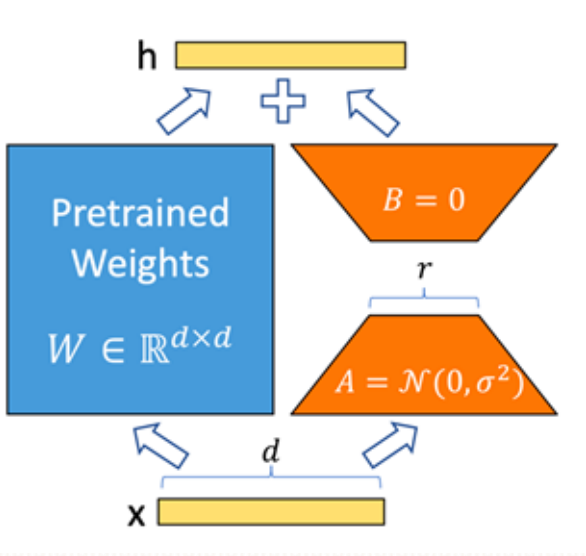
The matrix above has a column rank of 2, since column 3 is the first minus the second column

Low-Rank Adaptation (LoRA)

Intuition

Re-casting the weight updates into a lower dimensional space translates to a lower capacity model.

As long as the capacity needed to do well on the fine-tuning task is lower than the dimensionality of the pre-trained weights, setting **r < A, r < B** should be fine for model performance.



Source: https://arxiv.org/abs/2106.09685

Full Fine-Tune: 4096X4096, for LoRA (r=8): 65536

Low-Rank Adaptation (LoRA)

How well does it do?

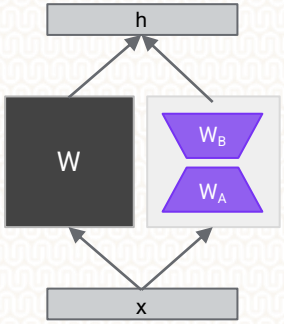
Model&Method	# Trainable Parameters	WikiSQL Acc. (%)	MNLI-m Acc. (%)	SAMSum R1/R2/RL
GPT-3 (FT)	175,255.8M	**73.8**	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 ($\text{Adapter}^{\text{H}}$)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 ($\text{Adapter}^{\text{H}}$)	40.1M	73.2	**91.5**	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	**91.7**	**53.8/29.8/45.9**
GPT-3 (LoRA)	37.7M	**74.0**	**91.6**	53.4/29.2/45.1

Table 4: Performance of different adaptation methods on GPT-3 175B. We report the logical form validation accuracy on WikiSQL, validation accuracy on MultiNLI-matched, and Rouge-1/2/L on SAMSum. LoRA performs better than prior approaches, including full fine-tuning. The results on WikiSQL have a fluctuation around ±0.5%, MNLI-m around ±0.1%, and SAMSum around ±0.2/±0.2/±0.1 for the three metrics.

Source: https://arxiv.org/abs/2106.09685

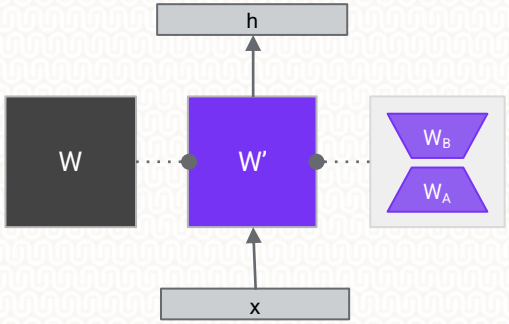
Low-Rank Adaptation (LoRA)

Keep LoRA separate



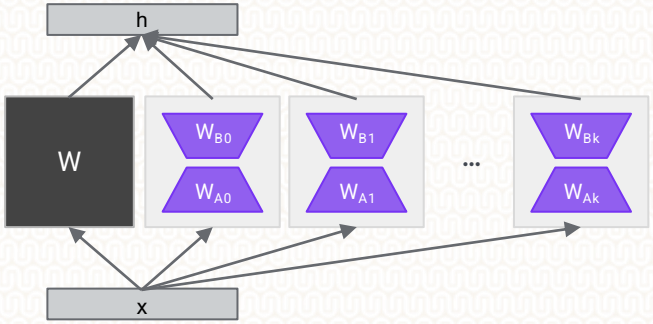
Incurs additional inference time

Merge the LoRA weights back to W

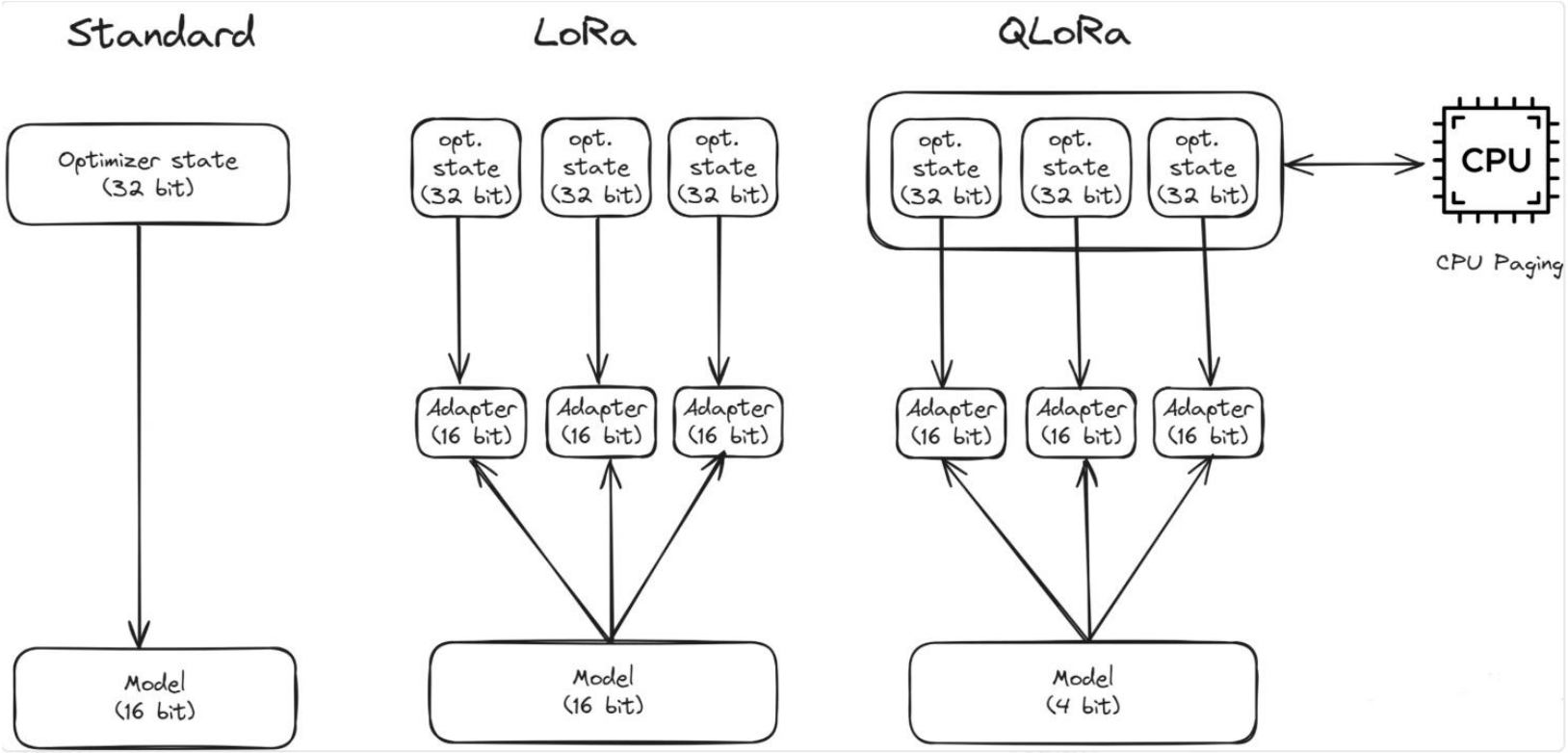


Inference time is equivalent to pretrained model

Maintain k LoRA weights per use case



Embrace additional inference time, maintain suite of specialized adaptors


Standard
Optimizer state
(32 bit)
Model
(16 bit)
LoRa
opt.
state
(32 bit)
opt.
state
(32 bit)
opt.
state
(32 bit)
Adapter
(16 bit)
Adapter
(16 bit)
Adapter
(16 bit)
Model
(16 bit)
QLoRa
opt.
state
(32 bit)
opt.
state
(32 bit)
opt.
state
(32 bit)
CPU
CPU Paging
Adapter
(16 bit)
Adapter
(16 bit)
Adapter
(16 bit)
Model
(4 bit)

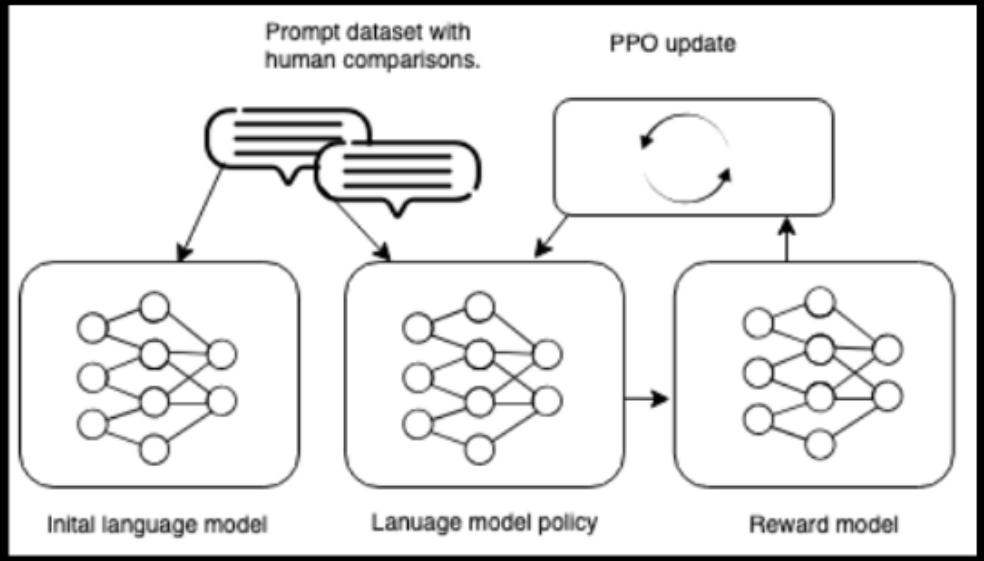
LoRA Variations

•**Low-rank adaptation (LoRA)** introduces low-rank matrices that interact with the original weights of the model, allowing for efficient updates to specific parts of the network without the need to retrain the entire model.

•**Weight-decomposed low-rank adaptation (DoRA)** decomposes the pre-trained weights into two components, magnitude and direction.

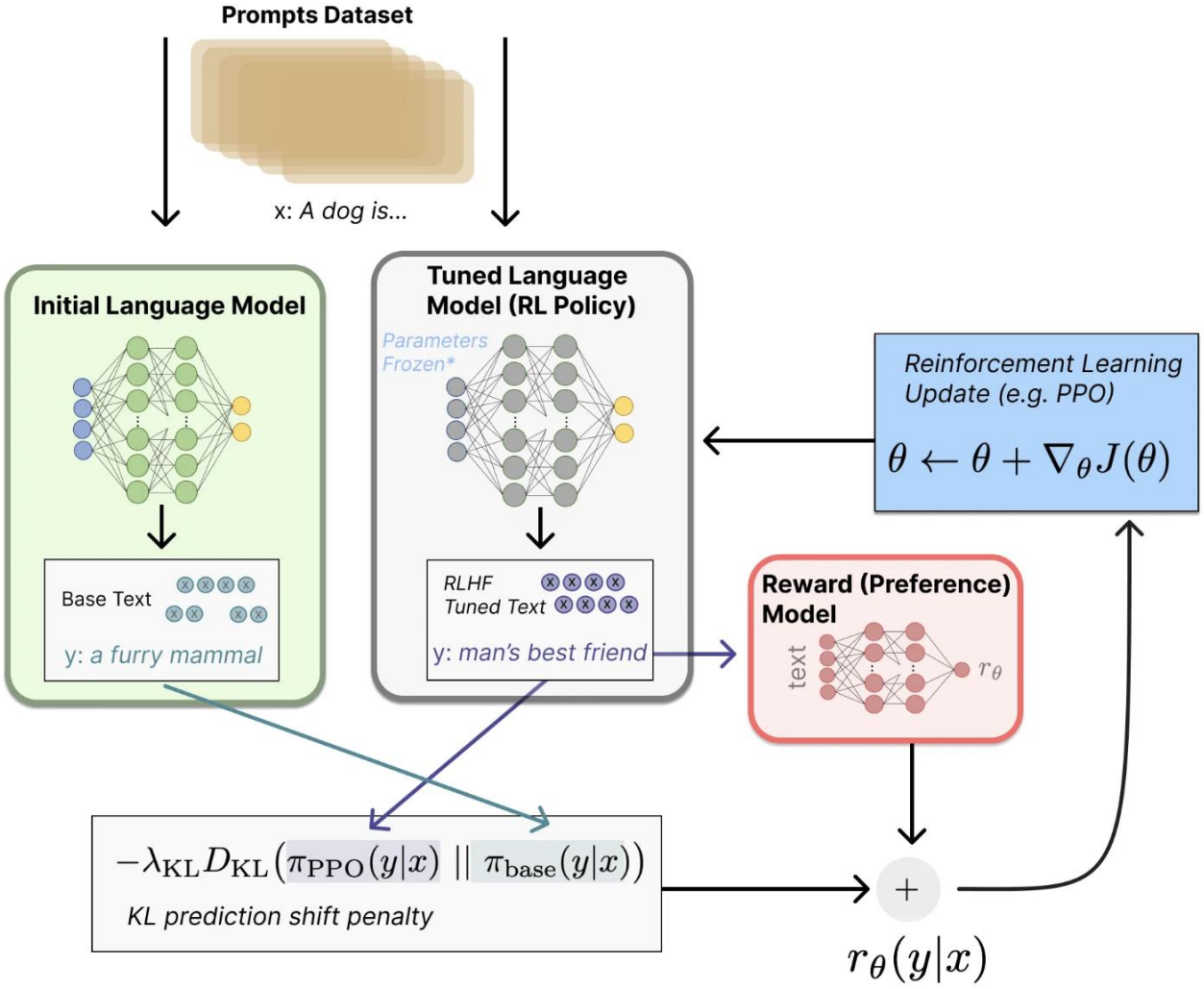
•**Quantized low-rank adapters (QLoRA)** introduces 4-bit NormalFloat (NF4), a new data type that is information-theoretically optimal for normally distributed weights, and double quantization to reduce the average memory footprint by quantizing the quantization constants.

RLHF (Reinforcement Learning from Human Feedback)



Simplified overview of a language model traing with RLHF.

Paper: Proximal Policy Optimization Algorithms", https://arxiv.org/abs/1707.06347 , Training language models to follow instructions with human feedback", https://arxiv.org/abs/2203.02155


Prompts Dataset
x: A dog is...
Initial Language Model
Base Text
y: a furry mammal
Tuned Language Model (RL Policy)
Parameters Frozen*
RLHF Tuned Text
y: man's best friend
Reinforcement Learning Update (e.g. PPO)
$\theta \leftarrow \theta + \nabla_\theta J(\theta)$
Reward (Preference) Model
text
r_θ
$-\lambda_{\mathrm{KL}} D_{\mathrm{KL}}\left(\pi_{\mathrm{PPO}}(y|x) \;||\; \pi_{\mathrm{base}}(y|x)\right)$
KL prediction shift penalty
+
$r_\theta(y|x)$

RLHF (Reinforcement Learning from Human Feedback)

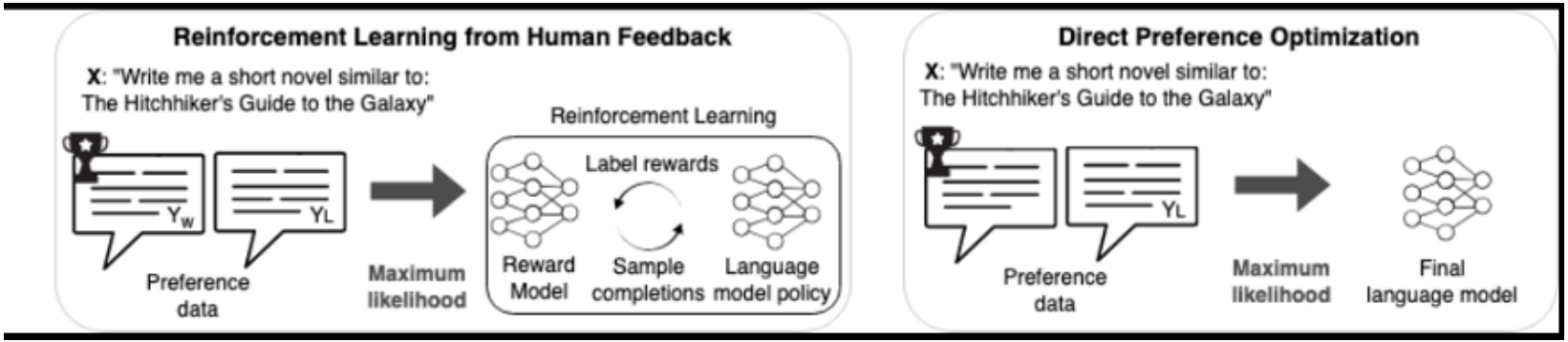
Human feedback provides an explicit supervisory signal that guides model behavior toward human-aligned objectives beyond likelihood-based training.

• **Value Alignment:** Preference-based feedback enables models to internalize human values, norms, and qualitative judgments that are difficult to encode as explicit labels.

• **Performance Refinement:** Iterative human evaluation helps identify systematic errors and failure modes, enabling continuous improvement through corrective optimization.

• **Behavioral Adaptability:** RLHF frameworks allow models to adjust to novel tasks, evolving requirements, and distribution shifts by incorporating ongoing human feedback.

Direct Preference Optimization (DPO)



DPO directly optimized based on chosen/rejected samples.

RLHF vs. DPO vs. GRPO

	RLHF	**DPO**	**GRPO** *(Group Relative Policy Optimization)*
Key Idea	Learn reward model, then optimize via RL	Directly optimize from preferences	Optimize policy using group-relative rewards
Reward Model	Required	Not required	Not required (implicit)
Optimization	PPO (reinforcement learning)	Supervised-style objective	Policy gradient with group normalization
Training Stability	Medium	High	High
Compute Cost	High	Low	Medium
Implementation	Complex	Simple	Moderate
Alignment Quality	Strong	Strong	Strong (relative preferences)
Best Use Case	Safety-critical alignment	Efficient instruction tuning	Ranking / group-based preference learning
Used By	OpenAI	Meta	DeepSeek, QWEN

Infrastructure Setup for Fine-Tuning

Model Selection

- LLaMA, Mistral, BERT, GPT-2, etc. (Hugging Face format compatible)
- Models must support continued training (freezable/unfreezable layers)

Dataset Requirements

Instruction fine-tuning format example:

```
{"instruction": "Translate to English",  "input": "你好",  "output": "Hello"}
```

Classification task format example:

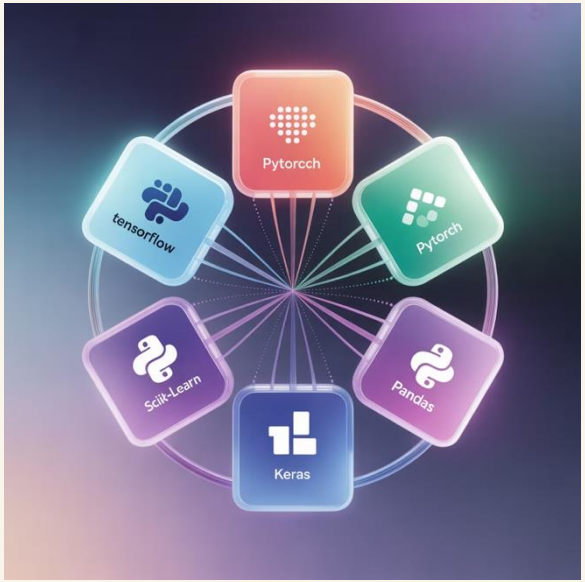
```
{"text": "This movie was excellent",  "label": 1}
```

Tokenizer & Padding Setup

- Must align with model's tokenizer (AutoTokenizer.from_pretrained(...))
- Configure with padding=True, truncation=True, max_length=N

Essential Libraries

- transformers, datasets, peft
- bitsandbytes (4-bit training)
- accelerate, trl (RLHF)
- wandb (optional for experiment tracking)



Bitsandbytes: Quantization for Efficient Training

What is bitsandbytes?

bitsandbytes is an open-source CUDA library for efficient **quantized neural network training and inference**, especially for large language models (LLMs). Created by **Tim Dettmers**, it's widely used in projects like **QLoRA** and Hugging Face Transformers.

Key Features

8-bit & 4-bit quantization

Reduces model size and memory usage by storing weights in lower precision (int8, int4) instead of float32

Linear layers replacement

Provides bnb.nn.Linear8bitLt as a drop-in replacement for torch.nn.Linear

QLoRA support

Enables training very large models (e.g., LLaMA-65B) on a single GPU with quantized weights

CUDA accelerated

All operations are GPU-accelerated using custom CUDA kernels

A **low-level efficiency backend**

Why is it important?

Imagine you have a 65B-parameter model. Running it in 16-bit precision may require > 350GB of VRAM. But with bitsandbytes, you can:

- ✅ Load it in **4-bit precision**
- ✅ Train or finetune on a **single A100 (80GB) GPU**
- ✅ **Retain >99% of the original model accuracy**



Training Samples

https://github.com/unslothai/unsloth?tab=readme-ov-file